AgroFácil · Deploy Blindado

Secure DevOps Tools & Cloud Computing · Global Solution 2026.2 · CitizenTech

Engenharia de Software · 4º ano (4ES) · FIAP

Professor: Gabriel Henrique Boyczuk Arcanjo

Integrantes

Nome completo	RM
Julia Ortiz	RM550204
Julia Palomari	RM551910
Leticia Baptista	RM550289
Vinicius Sobreira Borges	RM97767

Item	Link
MVP publicado (URL pública)	https://ca-agrofacil-prod.whitewave-37b28008.brazilsouth.azurecontainerapps.io
Repositório público	https://github.com/viniciusborgess/agrofacil
Pipeline (GitHub Actions)	https://github.com/viniciusborgess/agrofacil/actions
Execução barrada	run 42676002417
Execução corrigida	run 42692692285

São Paulo, 09/11/2026

1. Ficha Técnica do MVP

Campo	Conteúdo
Produto	AgroFácil: apoio à decisão do pequeno produtor rural. Módulo publicado: AgroFácil Visão.
Cidadão e território	Pequeno produtor rural, muitas vezes sem assistência técnica por perto.
Problema	A doença na folha é percebida tarde, e o produtor não sabe qual é o primeiro cuidado.
Proposta de valor	Apontar o celular para a folha e receber, em segundos, onde está o possível problema e o que fazer primeiro.
Componentes	Navegador → API FastAPI (Container Apps) → modelo YOLO11n em ONNX; Prometheus e Grafana.
Nuvem	Azure (Brazil South): Container Apps, Container Registry, Log Analytics, Storage, identidades gerenciadas.
Segurança	Gitleaks, Semgrep e Trivy a cada push; OIDC sem senha; HTTPS; sem porta de administração; container não root.
URL do MVP e repositório	https://ca-agrofacil-prod.whitewave-37b28008.brazilsouth.azurecontainerapps.io https://github.com/viniciusborgess/agrofacil
ODS	2 (agricultura sustentável), 9 (inovação), 10 (redução das desigualdades), 13 (ação climática).

2. Arquitetura

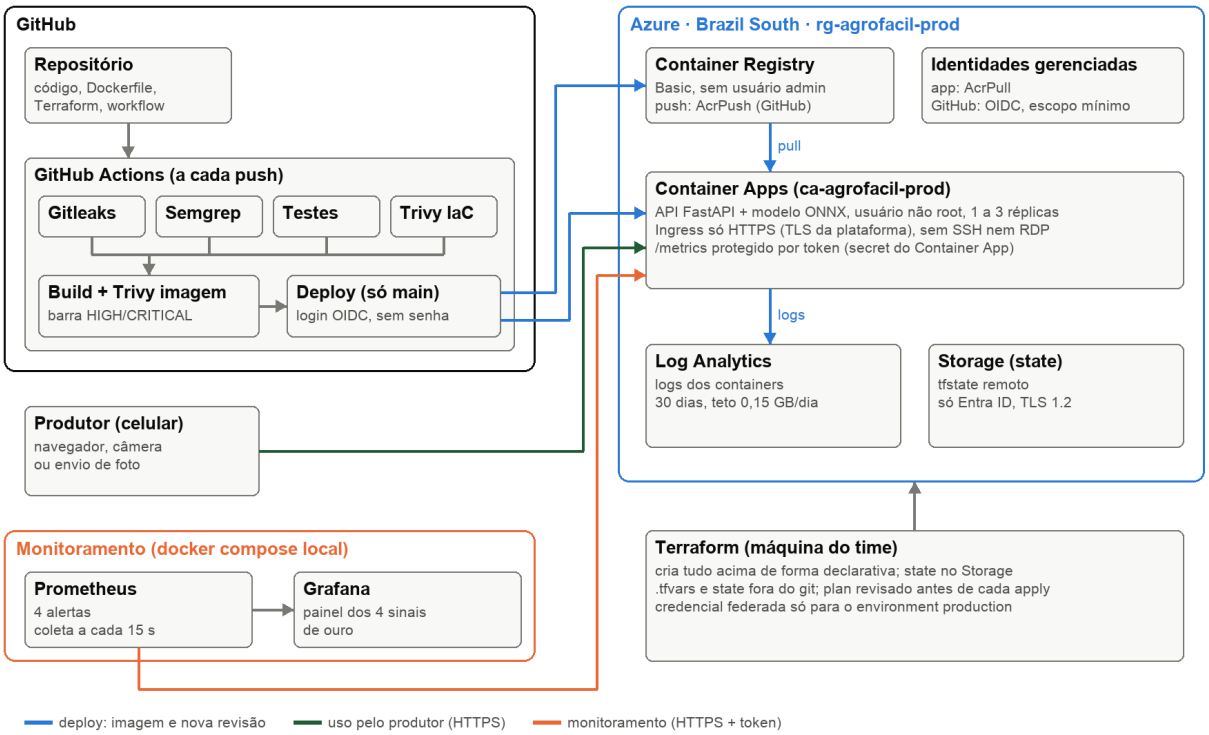


Figura 1. Arquitetura do deploy: o código passa por quatro barreiras de segurança antes do build, a imagem é verificada de novo e só então publicada no Azure por um login sem senha.

Escolhemos o Azure Container Apps, serviço gerenciado de containers, em vez de uma máquina virtual. Não há sistema operacional para manter nem porta SSH ou RDP para proteger: o único acesso é o ingress HTTPS da plataforma. A região Brazil South mantém os dados no país e reduz a latência para o produtor.

3. Container e infraestrutura como código

3.1 Dockerfile e docker compose

- Imagem python:3.11-slim com só as dependências de produção (sem PyTorch): FastAPI, ONNX Runtime e Pillow.
- Pacotes do Debian atualizados no build; setuptools e wheel removidos da imagem final, porque não rodam em produção.
- Processo roda como usuário sem privilégios (uid 1000), com HEALTHCHECK.
- Nenhum segredo na imagem: o token de métricas entra como variável de ambiente vinda de um secret da plataforma. O .dockerignore nega tudo e libera só o necessário.
- O docker compose sobe API, Prometheus e Grafana. A API roda com sistema de arquivos só leitura, sem capabilities e com no-new-privileges; as portas ficam presas a 127.0.0.1.

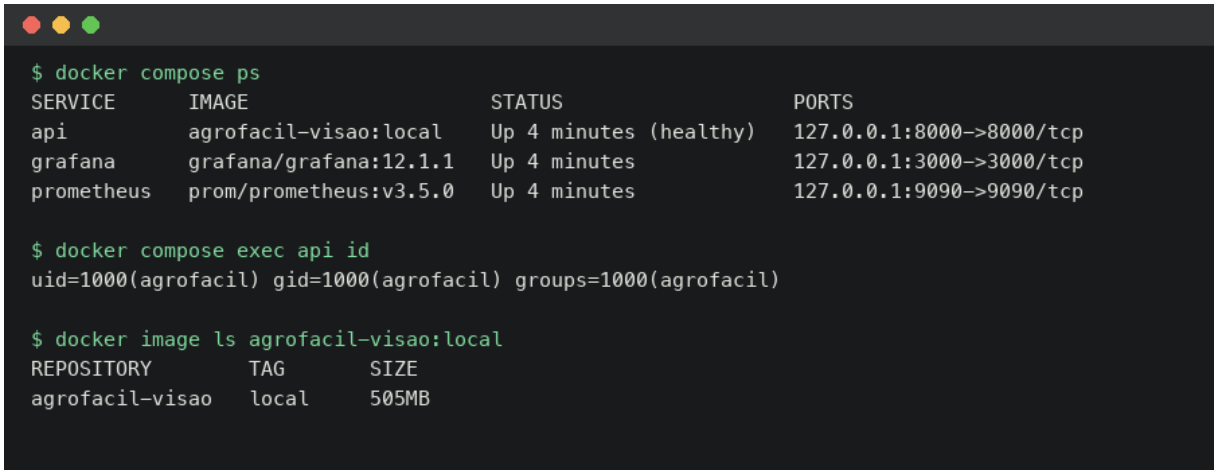


Figura 2. Stack local no ar com docker compose e o processo da API rodando com uid 1000, não root.

3.2 Terraform

Toda a nuvem é criada por Terraform (pasta infra/): grupo de recursos, registro de imagens, Log Analytics, ambiente e aplicativo do Container Apps, duas identidades gerenciadas com permissões mínimas e a credencial federada do GitHub. Cada apply foi precedido de um plan revisado; o plan atual não tem diferenças.

Exigência	Como atendemos
State fora do repositório	Backend remoto no Azure Storage, com acesso só por Entra ID (sem chave compartilhada), TLS 1.2 e sem acesso público. O arquivo backend.hcl é gerado por scripts/bootstrap_state.sh e fica fora do git.
.tfvars fora do repositório	terraform.tfvars está no .gitignore; versionamos só terraform.tfvars.example.
Porta de administração nunca aberta para 0.0.0.0/0	Não existe porta de administração: Container Apps não expõe SSH nem RDP. O único ingress é HTTPS na 443; HTTP recebe 301 para HTTPS. O registro de imagens não tem usuário admin.
Segredos	Token de métricas como secret do Container App; no GitHub só há identificadores do OIDC, não senhas.

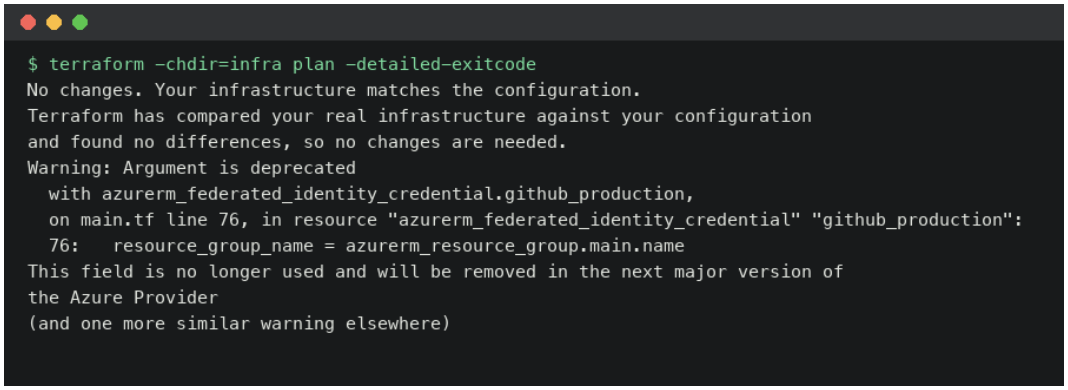


Figura 3. terraform plan sem diferenças: a nuvem corresponde ao código.

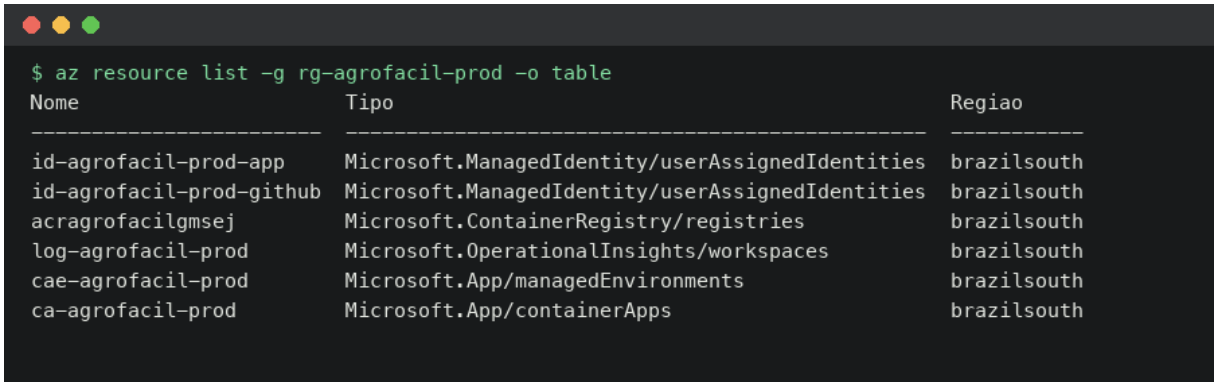


Figura 4. Recursos criados pelo Terraform no grupo rg-agrofacil-prod.

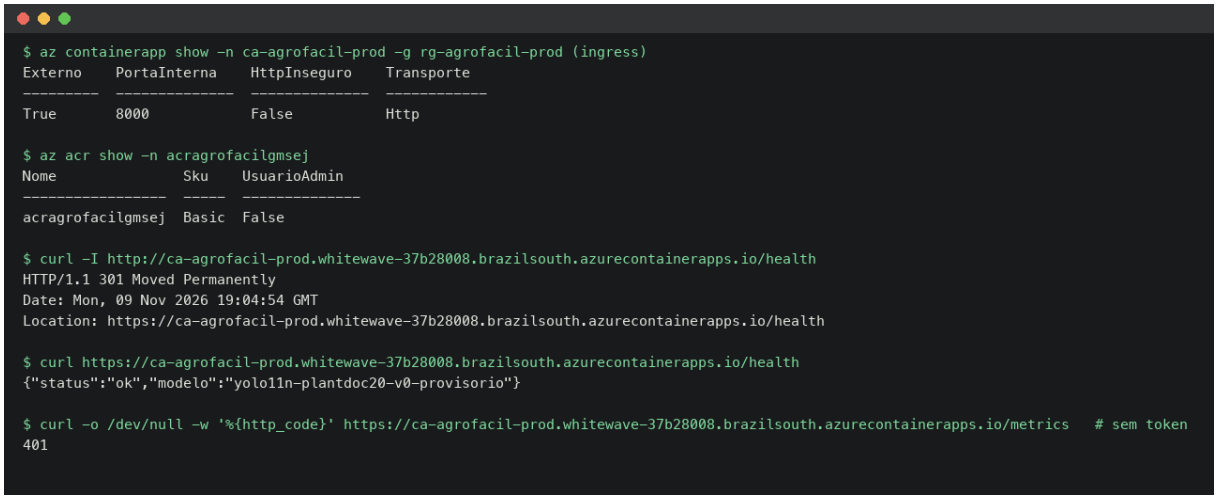


Figura 5. Ingress só HTTPS (HTTP é redirecionado), registro sem usuário admin, /health respondendo e /metrics recusando acesso sem token.

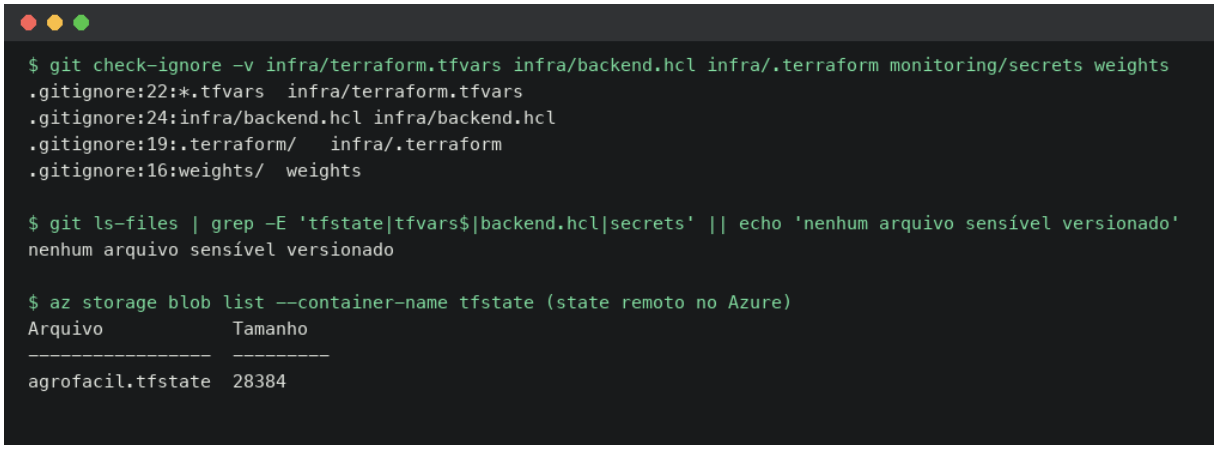


Figura 6. tfvars, backend e segredos ignorados pelo git; o state está no Azure Storage.

4. Pipeline DevSecOps

Etapa	Ferramenta	Barra quando
Segredos	Gitleaks 8.21.2 (histórico completo)	encontra credencial em qualquer commit
SAST	Semgrep (p/python, p/owasp-top-ten, p/dockerfile)	há achado de segurança no código
Testes	pytest (contrato da API, LGPD, limites)	algum teste falha
IaC	Trivy config (Terraform e Dockerfile)	má configuração HIGH ou CRITICAL
Imagem	docker build + Trivy image	vulnerabilidade HIGH ou CRITICAL com correção
Deploy	azure/login (OIDC) + az containerapp update	só roda em main, depois de tudo passar
Verificação	teste de fumaça em /health	a nova versão não responde

- Sem senha de nuvem no GitHub. Os GitHub Secrets guardam só os identificadores do OIDC. O Azure aceita o token apenas do environment production deste repositório, identificado pelos IDs imutáveis do dono e do repositório.
- Permissão mínima. A identidade do GitHub pode enviar imagens ao registro, ler o grupo e atualizar só o app.
- Cadeia de suprimentos. Cada Action está fixada por hash de commit, e Gitleaks e Trivy são baixados com SHA-256 conferido. O modelo de IA vem de uma Release do GitHub, também conferida por SHA-256 (model.lock.json).

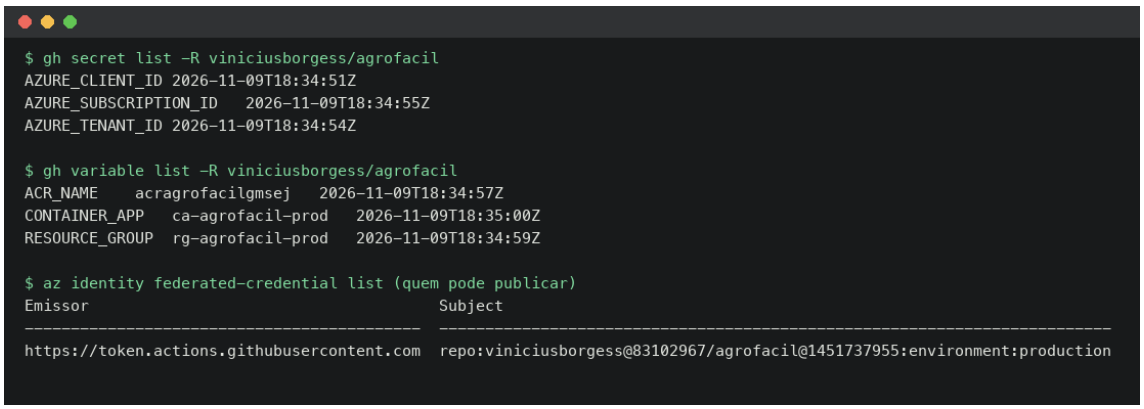


Figura 7. Secrets e variáveis do GitHub (valores ocultos) e o subject autorizado na credencial federada.

4.1 Execução barrada e execução corrigida

A barreira funcionou em um caso real, não simulado. No commit bb94d1a, os quatro primeiros estágios passaram, mas o Trivy encontrou na imagem vulnerabilidades HIGH e CRITICAL com correção disponível: estouro de buffer no Pillow 11.3 (CRITICAL), negação de serviço na Starlette 0.46, falhas no zlib do Debian e em setuptools e wheel herdados da imagem base. O deploy não executou.

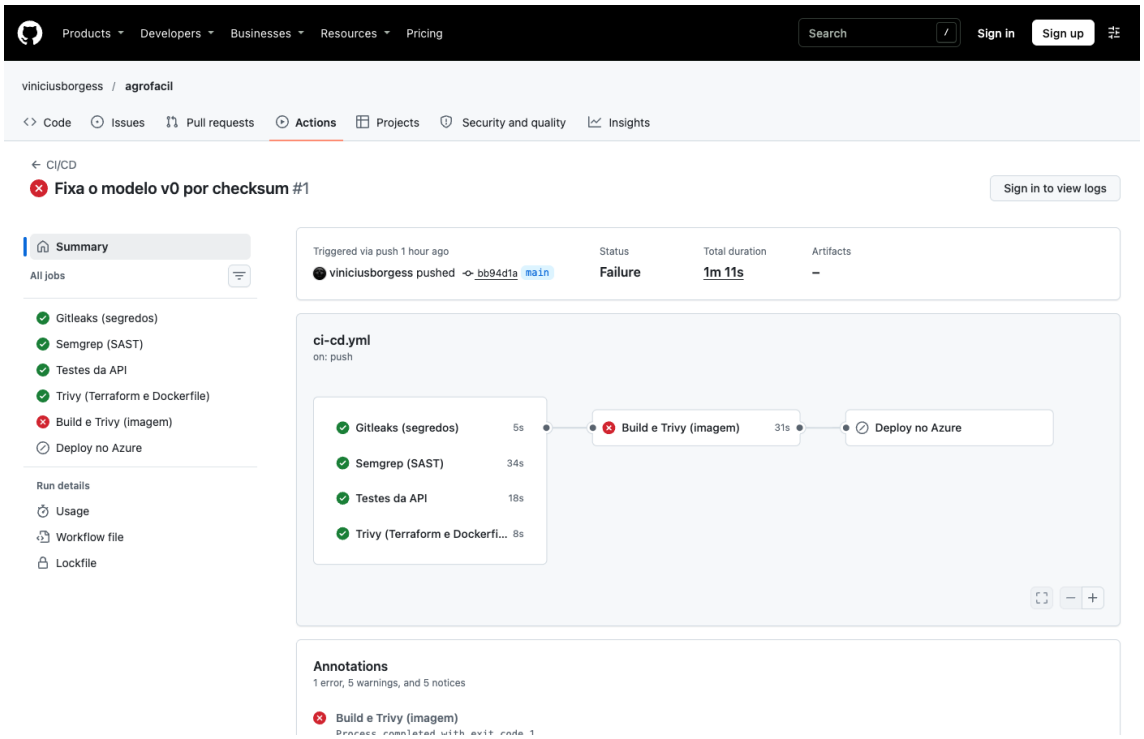


Figura 8. Execução barrada: o build falhou e o deploy foi pulado.

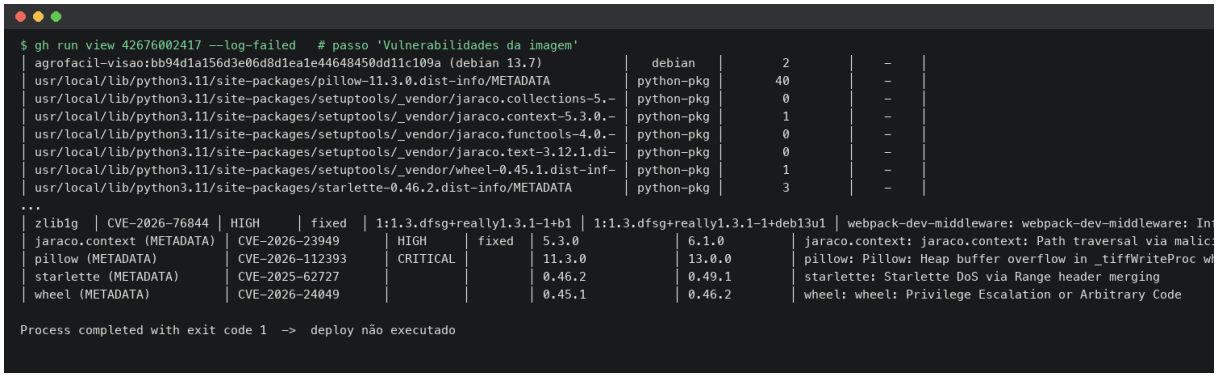


Figura 9. Achados do Trivy no log da execução barrada, com as versões corrigidas.

A correção atualizou Pillow para 13.0, FastAPI para 0.162 (Starlette 1.10), aplicou as atualizações do Debian no build e removeu setuptools e wheel da imagem. Na execução seguinte, todos os estágios passaram e o deploy publicou a nova versão sozinho.

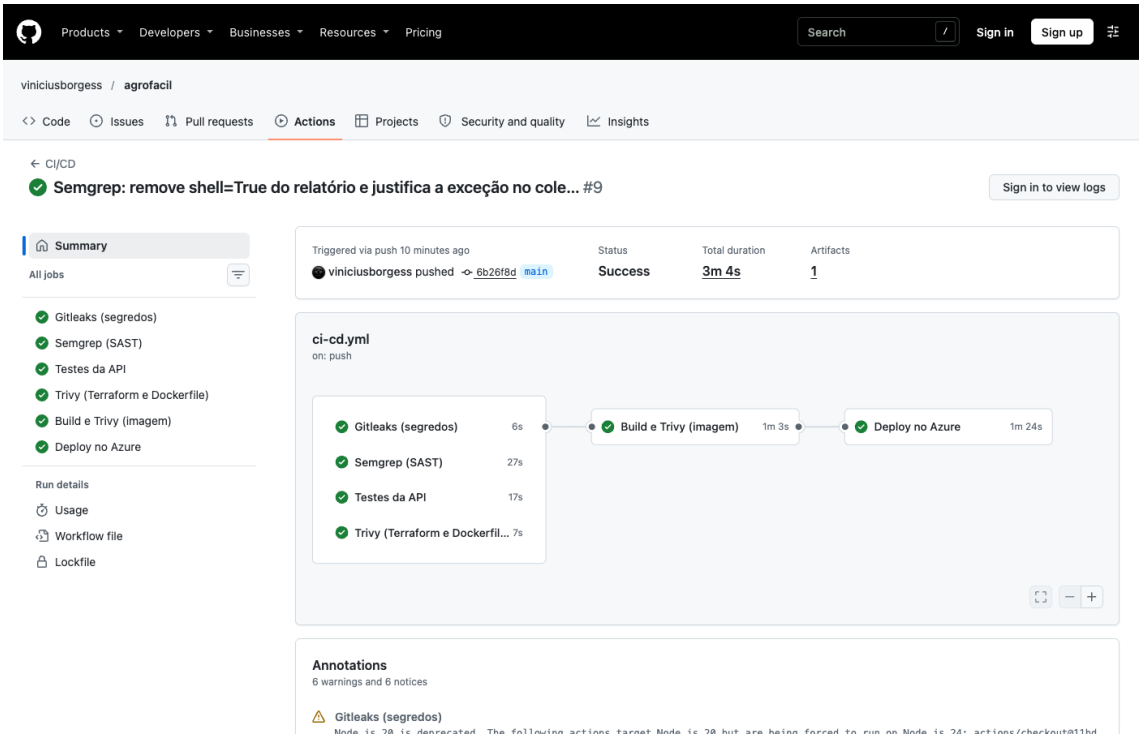


Figura 10. Execução corrigida: todos os estágios aprovados e deploy automático.

4.2 Segundo bloqueio: Semgrep

O pipeline barrou um segundo problema real. Os scripts que coletam as evidências deste relatório chamavam comandos com subprocess e shell=True, padrão que o Semgrep classifica como bloqueante por permitir injeção de comandos. A execução 42692489668 parou no SAST, e o build e o deploy foram pulados. Na correção, o gerador do relatório passou a chamar o Terraform sem shell. O coletor de evidências, que roda só na máquina do time e com comandos fixos no código, recebeu uma exceção explícita e justificada no próprio código. A execução 42692692285 passou em todos os estágios e publicou o modelo final.

```
$ gh run view 42692489668   # jobs
Testes da API: success
Trivy (Terraform e Dockerfile): success
Gitleaks (segredos): success
Semgrep (SAST): failure
Deploy no Azure: skipped
Build e Trivy (imagem): skipped

$ gh run view 42692489668 --log-failed   # Semgrep (SAST)
    scripts/build_devops_report.py
   ❯❯❱ python.lang.security.audit.subprocess-shell-true.subprocess-shell-true
          ❮❮ Blocking ❯❯
           31┆ return subprocess.run(command, shell=True, cwd=ROOT, capture_output=True,
    scripts/devops_evidence.py
   ❯❯❱ python.lang.security.audit.subprocess-shell-true.subprocess-shell-true
          ❮❮ Blocking ❯❯
           24┆ result = subprocess.run(command, shell=True, cwd=ROOT, capture_output=True, text=True)
  • Findings: 2 (2 blocking)
```

Figura 10b. Execução barrada pelo Semgrep: os achados bloqueantes impediram build e deploy.

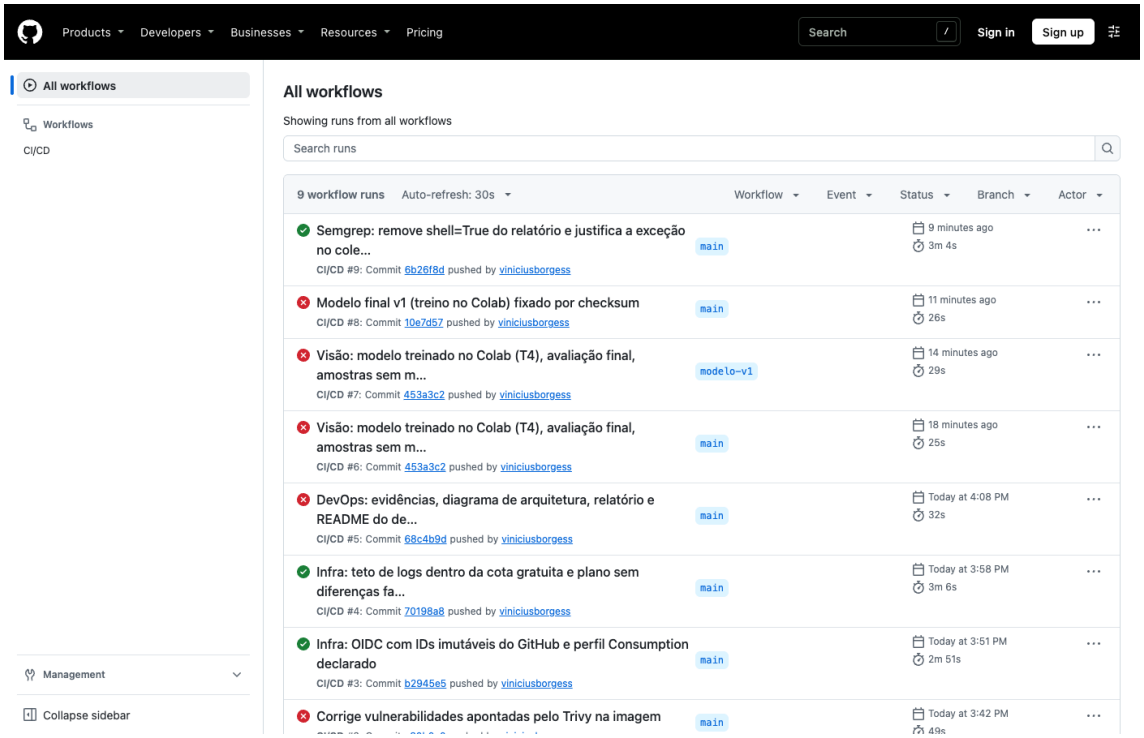


Figura 11. Histórico do pipeline.

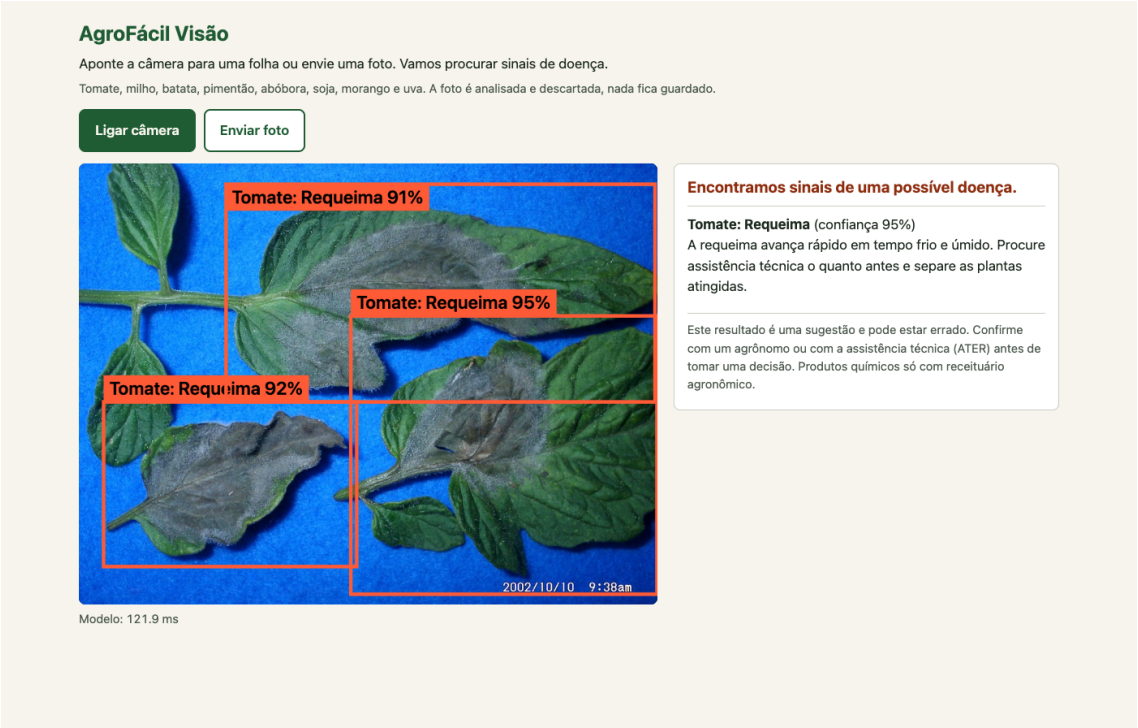


Figura 12. MVP em uso na URL pública (https://ca-agrofacil-prod.whitewave-37b28008.brazilsouth.azurecontainerapps.io).

5. Observabilidade

A API expõe métricas no formato do Prometheus em /metrics, protegido por token. O Prometheus roda localmente com docker compose e lê, por HTTPS, a API publicada no Azure, como a disciplina permite. O Grafana mostra os quatro sinais de ouro, e o Prometheus avalia quatro regras de alerta.

Sinal de ouro	Métrica	Alerta
Latência	http_request_duration_seconds (p50 e p95)	p95 da detecção acima de 1 s por 2 min
Tráfego	http_requests_total (req/s por rota)	-
Erros	http_requests_total com status 5xx e 4xx	mais de 5% de 5xx por 1 min
Saturação	inference_slots_in_use / inference_slots_max e recusas 503	mais de 0,5 recusa/s por 1 min
Disponibilidade	up	API fora do ar por 1 min



Figura 13. Painel do Grafana com os quatro sinais de ouro do MVP no Azure, durante o teste de carga.

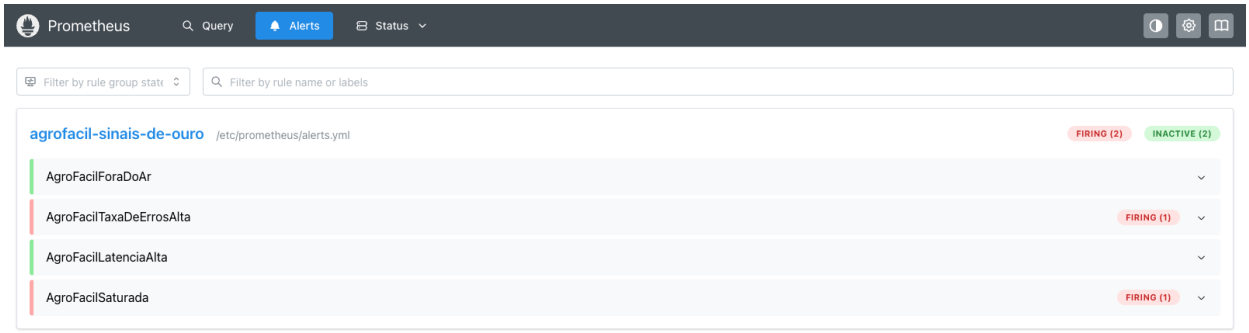


Figura 14. Alerta disparado no Prometheus durante o teste de carga.

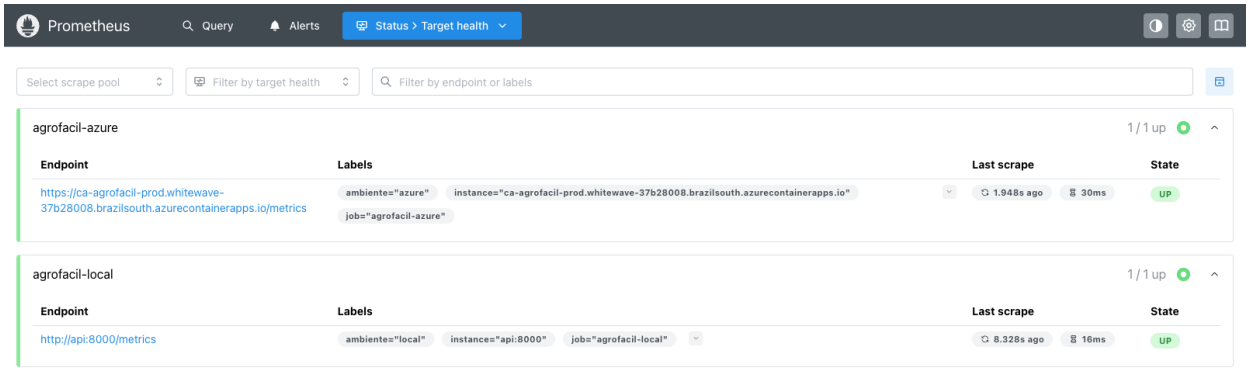


Figura 15. Alvos monitorados: API local e API no Azure.

Teste de carga (scripts/load_test.py): 24 conexões simultâneas por 240 s contra o MVP no Azure: 14.025 requisições (58 por segundo); 4.529 atendidas com sucesso e 9.492 recusadas com 503 pelo limite de 4 análises simultâneas por réplica, sem travar o serviço; latência p50 de 358 ms e p95 de 880 ms; os alertas AgroFacilSaturada e AgroFacilTaxaDeErrosAlta dispararam e voltaram ao normal após o teste.

6. Responsabilidade compartilhada

Camada	Azure	Equipe AgroFácil
Data center, rede física, hardware	Sim	-
Sistema operacional dos hosts e orquestração (Container Apps)	Sim	-
TLS, certificado e balanceamento do ingress	Sim	Exigir só HTTPS (configurado)
Imagem do container (base, bibliotecas)	-	Sim: Trivy a cada push e atualização das dependências
Código da aplicação	-	Sim: Semgrep, testes e revisão
Identidade e acesso (IAM)	Fornece Entra ID e RBAC	Sim: OIDC, identidades com permissão mínima
Segredos	Fornece secrets do Container App	Sim: nada no código, na imagem ou no git
Dados pessoais (LGPD)	Infraestrutura certificada	Sim: minimização, retenção e transparência
Monitoramento e resposta	Fornece métricas e logs da plataforma	Sim: painel, alertas e ação

7. Onde ficam os dados pessoais (LGPD)

Dado	Onde fica	Tratamento
Foto da folha	Só na memória do container, durante a análise	Descartada em seguida; nunca gravada em disco (teste automatizado)
Endereço IP do usuário	Logs de acesso da plataforma no Log Analytics	Retenção de 30 dias; acesso só da equipe via RBAC
Logs da aplicação	Log Analytics (Brazil South)	Só status, quantidade de detecções e tempo; sem imagem nem nome de arquivo
Métricas	Prometheus local	Agregadas, sem identificação de pessoa
Segredos e state	Secrets do Container App e Storage	Criptografados em repouso pela plataforma; fora do git

Todos os recursos estão em Brazil South, então os dados não saem do país. O app informa ao usuário que a foto é analisada e descartada.

8. Estimativa de custo mensal

Recurso	Uso esperado	Pior caso	Premissa
Container Apps (1 réplica, 0,5 vCPU, 1 GiB)	US$ 10,04	US$ 33,48	Ocioso na maior parte do tempo; o pior caso é processar sem parar o mês inteiro
Container Registry Basic	US$ 5,00	US$ 5,00	Diária fixa; 10 GB incluídos
Log Analytics	US$ 0,00	US$ 0,00	Teto de 0,15 GB/dia, dentro dos 5 GB/mês gratuitos
Storage do state do Terraform	< US$ 0,01	< US$ 0,01	Um arquivo de cerca de 30 KB
Identidades, ambiente, TLS e domínio	US$ 0,00	US$ 0,00	Sem cobrança
Total	US$ 15,04	US$ 38,48	

Preços de varejo do Azure em USD para brazilsouth, consultados na API oficial (prices.azure.com) em 09/11/2026, e salvos em docs/devops/precos_azure.json. A cota gratuita mensal do Container Apps (180 mil vCPU-s e 360 mil GiB-s) foi descontada. Com min_replicas = 0 o app custaria perto de zero, ao preço de alguns segundos de espera na primeira requisição; mantivemos 1 réplica para a correção. O custo sai dos créditos do Azure for Students.

9. Como reproduzir

1. scripts/bootstrap_state.sh: cria o Storage do state e o infra/backend.hcl.
2. Copie infra/terraform.tfvars.example para infra/terraform.tfvars e preencha; depois terraform -chdir=infra init -backend-config=backend.hcl e plan/apply (primeiro sem image_tag).
3. Cadastre no GitHub os secrets AZURE_CLIENT_ID, AZURE_TENANT_ID e AZURE_SUBSCRIPTION_ID e as variáveis ACR_NAME, RESOURCE_GROUP e CONTAINER_APP (valores nas saídas do Terraform).
4. Faça push em main: o pipeline envia a imagem. No primeiro deploy, rode o apply com image_tag igual ao commit para criar o app.
5. Monitoramento: scripts/monitoring_setup.sh e docker compose up -d; painel em http://localhost:3000.

O passo a passo completo está no README do repositório.